Li Lab Expanse Info/Docs

Expanse help desk email: consult@sdsc.edu
Feel free to add/edit sections on this living document!

I. Project Basic Info

Path to project allocations:

- Hojun: /expanse/lustre/projects/csd933
- Nick: /expanse/lustre/projects/csd922

Data Storage:

- Your home directory should never be used for storing large data!
    - Ex: For nick, don't store data at ~/ or //home/nic002
- Store large data under your project allocation path
    - Ex: For nick, store data in any directory under /expanse/lustre/projects/csd922

Starting an interactive session

- For quickly using conda, I recommend using the "debug node" or an interactive session
- An interactive session puts you onto a compute node rather than the login node, giving you more compute power and RAM for activities such as file manipulation and installation
- Ex:

```
srun --partition=debug --pty --account=csd832 --nodes=1
--ntasks-per-node=1 --mem=2G -t 00:30:00 --wait=0 --export=ALL
/bin/bash
```

- Change partition to run a longer interactive session - see for other partitions

Path to Expanse Scratch (purged 90 days from creation date):

- /expanse/lustre/scratch/$USER/temp_project

Path to conda environments:

- In general: /home/{user}/expanse/miniconda3/envs/{name of environment}
- Ex: /home/nic002/expanse/miniconda3/envs/seurat

Viewing all possible modules:

- module spider

Info on how to load a specific module:

- module spider <module>

II. VS Code Server setup (Optional)

A. I use VS Code's ssh extension to browse through Expanse most of the time
B. If you get stuck on the part where VS Code downloads the vscode server to expanse, check this out https://stackoverflow.com/questions/56073276/how-can-i-install-vs-code-server-manually-and-tell-vs-code-remote
    1. The problem you are likely stuck on is due to the login node not having enough memory to complete the operation
    2. To manually install the server, start up a debug or interactive session and follow the steps in the stack overflow
C. Also see this:
    1. https://github.com/microsoft/vscode-remote-release/issues/11027
    2. VS Code Remote seems to run into a memory error… revert from version 1.101 to **1.100.0** (I think even 1.100.3 won't work)
    3. For download links like https://update.code.visualstudio.com/**1.100.3**/win32-x64-user/stable simply change the version to https://update.code.visualstudio.com/**1.100.0**/win32-x64-user/stable
    4. SUPER IMPORTANT AFTER INSTALLATION: Turn off auto updating for vscode… otherwise the second you quit out of vscode and reopen it, your version will auto update

III. Environment Information (See next section for actual setup)

A. MINICONDA

There is a default anaconda environment on Expanse where we could use it to create conda environments and download packages. However, this is **Not Recommended**. The SDSC encourages us to install our own **MINICONDA** in our **home** directory (make sure the path of your current directory where you manage environments should always contain 'home' rather than 'lustre' in it).

- Here's the link that I used to install miniconda on Expanse, select 'Linux-64 bit' and follow the cml under 'Linux' tab:
  https://docs.conda.io/projects/miniconda/en/latest/
- Make sure that the ~/.bashrc is modified to specify the correct environment.

After installing miniconda, you could individualize your own environment using commands below. It's suggested by the IT Group that we build separate environments for R and Python packages if you're gonna use these two at the same time. KEEP each env **LIGHT and SMALL**. If you tried to install packages from both R and Python in the **same** environment, it could cause problems, for example getting stuck at the environment solving stage forever. So they suggested to manually load the R and Python in job shell scripts using the following commands:

- conda activate ${your_R_env} # activate your R env

- conda activate ${your_Python_env} # activate your Python env
- conda activate ${your R_env} # switch back to R if needed
- I noticed that normally we need to deactivate to activate another env, but it could be skipped in shell script.

One thing we need to notice is that after creating our own environments and running jobs in them, we need to specify the corresponding 'module load' commands in our shell scripts.

- The commands that could work is:

```
module reset
module load cpu/0.15.4
module unload anaconda3/2020.11
source ~/.bashrc
conda activate ${your_env_name}
```

B. MAMBA (Optional)

If you find Conda is not working quite well managing packages within environments, you could install **Mamba** in your base environment and that should be the only package/tool being installed in your base environment.

```
conda install -n base --override-channels -c conda-forge mamba 'python_abi=*=*cp*'
```

Then, you could try install your packages using the same commands as conda and just replace all 'conda' with 'mamba'

C. YAML Files

Do the following to add config to all your conda environments:

```
vim ~/.condarc
```

Make sure your file looks like this:

```
Channels:
    - conda-forge
    - bioconda
    - nodefaults
channel_priority: strict
```

YAML files should be maintained carefully to avoid any conflicts. For each package you install, you need to check the version of R/Python it relies on and if that is compatible with other packages that use R/Python. For Python packages, use this website: https://pypi.org/.

- Conda channels we should use:
  - conda-forge and bioconda
  - Try not to use 'Anaconda', 'r', and 'defaults' channels since they may keep old versions of packages and not up-to-date, not comprehensive
  - Channels listed in Anaconda conflicts with conda-forge

- Channel order is very **important**!!!
  - conda config --add channels conda-forge
  - conda config --add channels bioconda
  - Bioconda relies on conda-forge
- Best practice: create env.yml files **manually**

```
channels:
    conda-forge
    bioconda
    nodefaults
dependencies:
    bioconda::pysam==0.19.1
    conda-forge::numpy==1.26.0
    conda-forge::python ==3.11.0
```

D. Do NOT use PIP or BREW on the server

IV. Installing Conda environment

- **Description of what conda environments are**: "Environments in conda are self-contained, isolated spaces where you can install specific versions of software packages, including dependencies, libraries, and Python/R versions. This isolation helps avoid conflicts between package versions and ensures that your projects have the exact libraries and tools they need." - source
- **Setting up conda on expanse**: I prefer using miniconda (lightweight conda installation) on expanse with mamba installed in the base environment. This keeps conda simple and fast while having the option to install packages efficiently with mamba.
  - Miniconda
    - Install script should install miniconda in home dir
- To install libraries for python or to set up instances of R in Expanse, it is recommended to use conda environments
  - To run code (R, Python, etc.) through Jupyter notebooks, conda environments are essential
- **Let's use an interactive session to install miniconda + getting conda base env running**:

```
#### START
# Upon logging into expanse, you are put on a login node. In this example I am on the
```

```
node “login01”. This node is really only useful for running scripts that you have and
regular cli commands like “ls” and “cd”. I don’t recommend anything compute heavy!.
[nic002@login01 ~]$

# You should start up an interactive session (Note: I think only 2 ppl can access the
“debug” partition under a single allocation at a time). This debug session is hard
limited to 1 cpu and 30 mins (im not sure if there’s a mem limit).
[nic002@login01 ~]$ srun --partition=debug --pty --account=csd933 --nodes=1
--ntasks-per-node=1 --mem=2G -t 00:30:00 --wait=0 --export=ALL /bin/bash

# Feel free to save this long command to an alias to make life easier… ask chat gpt
how to do this, I swear it’s worth! Now running “sdebug” should start up debug session
for you.
[nic002@login01 ~]$ echo “alias sdebug='srun --partition=debug --pty --account=csd933
--nodes=1 --ntasks-per-node=1 --mem=2G -t 00:30:00 --wait=0 --export=ALL /bin/bash'"
>> ~/.bashrc
# Reload ~/.bashrc so the alias works immediately

[nic002@login01 ~]$ source ~/.bashrc
# Run alias
[nic002@login01 ~]$ sdebug

# Notice that I am on a new node “exp-9-55” and not “login01”
[nic002@exp-9-55 ~]$

# Switch over to your home dir
[nic002@exp-9-55 ~]$ cd ~

# Make a dir for miniconda. This will be the location of your conda install
[nic002@exp-9-55 ~]$ mkdir -p ~/miniconda3

# Upon download the miniconda install script using wget
[nic002@exp-9-55 ~]$ wget
https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O
~/miniconda3/miniconda.sh

# Run the install script! Note that if you have a permission issue, try running “chmod
770 ~/miniconda3/miniconda.sh” to make the script executable
[nic002@exp-9-55 ~]$ bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3

# Remove the install script
[nic002@exp-9-55 ~]$ rm ~/miniconda3/miniconda.sh

# Now that you have conda installed, you will notice that typing conda isn’t
immediately found
[nic002@exp-9-55 ~]$ conda
> bash: conda: command not found

# You will need to run the following command to start up conda.
[nic002@exp-9-55 ~]$ source ~/miniconda3/etc/profile.d/conda.sh
[nic002@exp-9-55 ~]$ conda activate
```

```
# This is also a great command to save to an alias!
# Add the alias to ~/.bashrc
[nic002@exp-9-55 ~]$ echo "alias conda_activate='source
~/miniconda3/etc/profile.d/conda.sh && conda activate'" >> ~/.bashrc
# Reload ~/.bashrc so the alias works immediately
[nic002@exp-9-55 ~]$ source ~/.bashrc
# Run alias
[nic002@exp-9-55 ~]$ conda_activate

# You should now be in your base conda env. You should now see output if you run
“conda”
(base) [nic002@exp-9-55 ~]$
#### END
```

- **Let’s make an environment for starting up jupyter lab sessions (See Interactive Computing session for how to use this env):**

```
#### START
# Make sure you have conda activated on an interactive session/node
(base) [nic002@exp-9-55 ~]$

# Create an environment named “jupyter” with jupyter, notebook, and jupyterlab
installed:
(base) [nic002@exp-9-55 ~]$ conda create -n jupyter jupyter notebook jupyterlab
#### END
```

- **Mamba installation (optional)**: Mamba is pretty much the ONLY thing that you should install on your base environment. Make sure every other installation is done on the environment that you intend to use.
    - Mamba works well most of the time, however I’ve had to use regular conda installs when mamba bugged out

```
#### START
# Make sure you have conda activated on an interactive session/node
(base) [nic002@exp-9-55 ~]$

# Install mamba on your base environment
(base) [nic002@exp-9-55 ~]$ conda install mamba
#### END
```

V. Interactive Computing

- **Jupyter Lab**:
    - To start a jupyter lab instance on expanse, I use Expanse’s galyleo script (galyleo is usable by default on expanse)
    - **How galyleo compares to regular jupyter notebooks:** Running the ***galyleo*** command is like running the ***jupyter notebook*** command for starting a jupyter notebook instance locally. The main difference is that galyleo will open a jupyter instance that you can access externally (same

or different wifi connection … at home or away) instead of locally (same wifi connection … at home)
    - **Recommended method for long term**: Once you get conda working you can give galyleo your conda init script path and a conda env to use upon start up. I recommend making a conda env with just **jupyter** and **jupyter lab** installed as this env will really just be used for starting up your jupyter lab session.
- **Let's create launch script for jupyter lab session:**
    - This will enable us to run python and R through notebooks

```
#### START
# Create the galyleo launch script
[nic002@login02 nic002]$ cat << 'EOF' > galyleo_launch_jupyter.sh
export PATH="/cm/shared/apps/sdsc/galyleo:${PATH}"
galyleo launch --account csd933 --partition shared --cpus 16 --memory 64 --time-limit
04:30:00 --conda-env jupyter --conda-init ~/miniconda3/etc/profile.d/conda.sh >
galyleo_launch.out
EOF

# Make it executable (optional but good practice)
chmod +x galyleo_launch_jupyter.sh
# I printed out the script's contents with cat. The script uses "galyleo", but now
instead of using anaconda3/2021.05 we now use an env called "jupyter" (see instructions
for installation above) and our conda init script (~/miniconda3/etc/profile.d/conda.sh)
which is created automatically when you install conda (~/ is kinda like an alias for your
home dir which is actually at /home).
[nic002@login02 nic002]$ cat galyleo_launch_jupyter.sh
export PATH="/cm/shared/apps/sdsc/galyleo:${PATH}"
galyleo launch --account csd933 --partition shared --cpus 16 --memory 64 --time-limit
04:30:00 --conda-env jupyter --conda-init ~/miniconda3/etc/profile.d/conda.sh >
galyleo_launch.out
[nic002@login02 nic002]$ ./galyleo_launch_jupyter.sh
#### END
```

● Alternative commands for quick startup IF YOU DON'T HAVE CONDA:

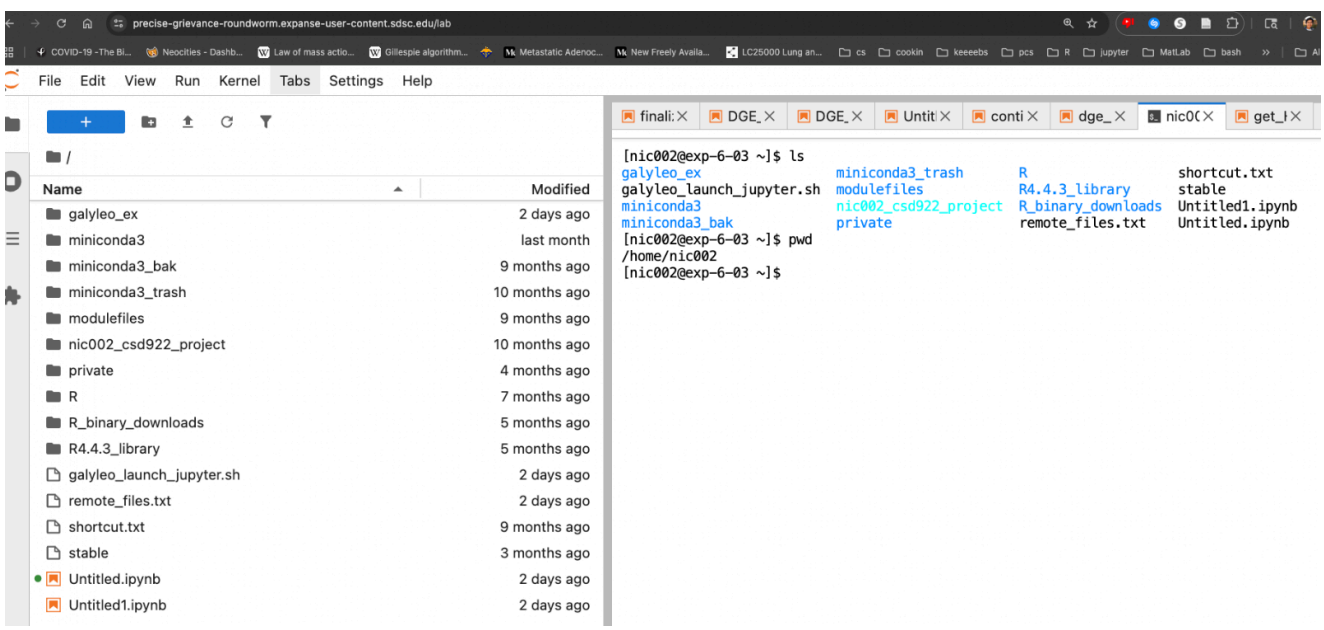
- **Note: Switch to the long term method ASAP (see code cell above)!** If you have no conda envs (the "jupyter" conda env) yet, try this, however try to get conda up and running because using the module anaconda3/2021.05 can slow down user experiences for everyone on the server

```
#### START
# First I switch over to my directory on hojun's allocation csd933
[nic002@login02 nic002]$ pwd
/expanse/lustre/projects/csd922/nic002

# In this directory I made a script called galyleo_launch_jupyter.sh  which you can copy
[nic002@login02 nic002]$ ls
galyleo_launch_jupyter.sh
```

```
# I printed out the script's contents with cat. The script essentially uses "galyleo" which is a custom script that we can use on expanse for starting up a jupyter labs session. In this script I use 16 cpus and 64 G of ram for 4 hours 30 mins.
[nic002@login02 nic002]$ cat galyleo_launch_jupyter.sh
export PATH="/cm/shared/apps/sdsc/galyleo:${PATH}"
galyleo launch --account csd933 --partition shared --cpus 16 --memory 64 --time-limit 04:30:00 --env-modules cpu/0.17.3b,anaconda3/2021.05

[nic002@login02 nic002]$ ./galyleo_launch_jupyter.sh
#### END
```

- Warning: If you are using this method a lot for resource intensive tasks, Mahidar from the Expanse IT team might send u an angry email about bogging down the shared file system due to your use of Expanse's conda installation (anaconda3/2021.05)

VI. After starting a Jupyter Lab session, you may only have file access in your home directory: (Make a symlink)

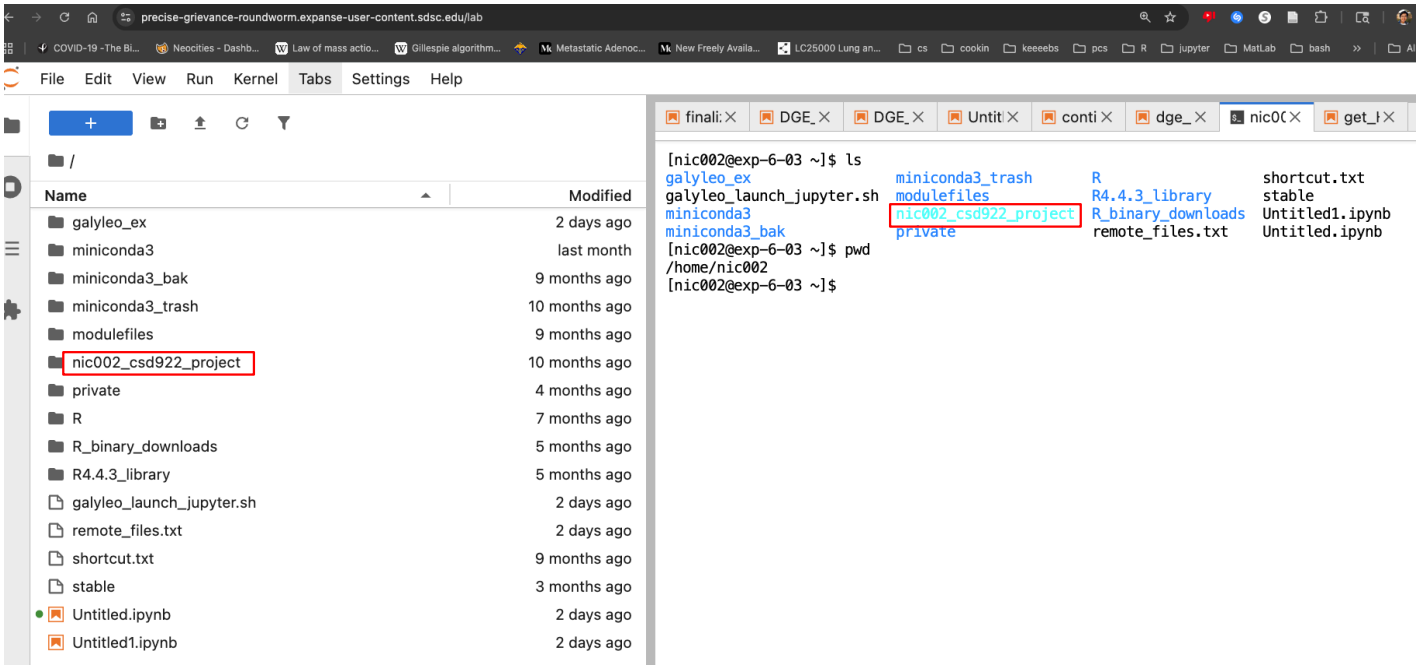


Upon the galyleo launch script finishing, you will likely be placed in your home directory, in my case /home/nic002, depending on where your launch script is located.

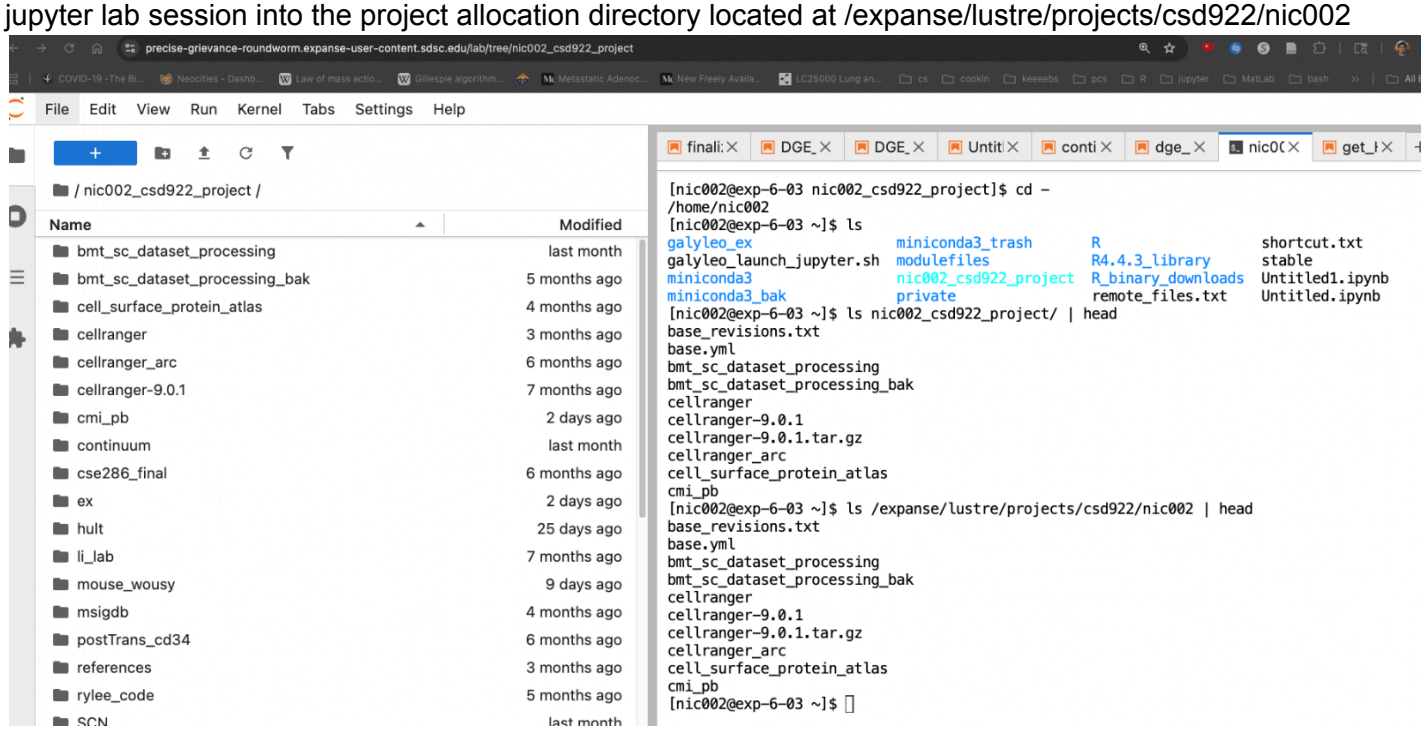
- This is bad when we have large data stored in a place like a project allocation because we will be locked to the home directory
  - You will notice that
- **A solution** to this is to make a "symbolic link" in the home directory to a directory where data can be accessed such as your project allocation directory

```
#### START
# Step 1: In your home directory create a link to your project allocation directory
(base) [nic002@exp-9-55 ~]$ ln -s /expanse/lustre/projects/csd922/nic002 nic002_csd922_project
#### END
```

- **Explanation:**
  - **ln -s → creates a symbolic link**
  - **/expanse/lustre/projects/csd922/nic002 → the target directory you want to link to**
  - **link_csd922_nic002 → the name of the link in your current working directory**



Notice that nic002_csd922_project is now a link in the home directory clicking on the link will bring the jupyter lab session into the project allocation directory located at /expanse/lustre/projects/csd922/nic002



Notice that the link matches typing out the whole path when ls'ing them.

- **Another solution** is simply to place your galyleo start script in your project allocation directory

VII. Ipykernel for Using Conda Environments in Jupyter

- **If you would like Jupyter to use a conda environment (ex you have an environment with pandas, sklearn, and seaborn installed) you must install and register Ipykernel on it**

```
#### START
# Step 1: Activate your conda environment
# Replace 'myenv' with the name of your actual environment
(base) [nic002@exp-9-55 ~]$ conda activate myenv

# Step 2: Install ipykernel into the active environment
# This allows the environment to function as a Jupyter kernel
(myenv) [nic002@exp-9-55 ~]$ conda install ipykernel

# Step 3: Register the environment as a Jupyter kernel
# --name is the kernel's internal identifier
# --display-name is what shows up in the Jupyter interface
(myenv) [nic002@exp-9-55 ~]$ python -m ipykernel install --user --name=myenv
--display-name "Python (myenv)"

# Step 4 (Optional): Launch Jupyter Notebook or Lab to check
# You should now see "Python (myenv)" as an option.
# Run galyleo launch script and click on URL printed to associated .out file
(myenv) [nic002@exp-9-55 ~]$ ./galyleo_launch_jupyter.sh
#### END
```

VIII. IRkernel Conda Environment for Using R in Jupyter

- **If you would like Jupyter to run R code in a notebook kind of setting, check out IRkernel**
  - This also allows you to conda install r packages to your IRkernel environment, however it is heavily recommended to install R libraries the normal way through install.packages() first
- **Special note: Conda envs with IRkernel shouldn't need Ipykernel**
- IRkernel works a lot like ipykernel which allows you to use conda environments as a kernel in a jupyter instance
- Installation steps
  - Activate a conda environment (with an R installation) that you would like to install IRkernel (let's say you created an environment called r442 using conda create -n r442 r-base=4.4.2)
  - Run conda commands:
    - conda install conda-forge::r-irkernel
  - Open up R on command line and run the following:
    - install.packages('devtools')
    - install.packages('IRkernel')
    - IRkernel::installspec(name = r442, displayname = 'R 4.4.2')
    - See the IRkernel github for more information
- Using Libraries from different environments or install locations
  - R libraries are only dependent on the R version which they were built on, meaning that as long as you have a compatible R version, you can use the same library folder between different environments

        - See .libPaths() for more details on adding different library directories to an R instance's library path.
    - Real ones know: .libPaths(c("~/miniconda3/envs/r442/lib/R/library/" , .libPaths()))
- **Let's create an env with R and install IRkernel**

```
#### START
# ---------------------
# Bash: Create and activate R conda environment, then start R
# ---------------------
# Make sure you have conda activated on an interactive session/node
(base) [nic002@exp-9-55 ~]$ conda create -n bio_r_py r-base r-essentials -y
(base) [nic002@exp-9-55 ~]$ conda activate bio_r_py
(bio_r_py) [nic002@exp-9-55 ~]$ R

# ---------------------
# R: Install IRkernel and register it with Jupyter
# ---------------------
install.packages("IRkernel")
IRkernel::installspec(name = "bio_r_py", displayname = "R (bio_r_py)")
q()

# ---------------------
# Bash: Launch Jupyter to check R kernel
# ---------------------
(base) [nic002@exp-9-55 ~]$ ./galyleo_launch_jupyter.sh
#### END
```

IX. Running Scripts Through SBATCH

- **Example R script that we want to run using Expanse (Sorry this isnt a great example script):**

```
#### File name: example.R
#### START
# Load in libraries
.libPaths( c( "~/miniconda3/envs/r442/lib/R/library/" , .libPaths() ) )
source("/home/nic002/nic002_csd922_project/bmt_sc_dataset_processing/scripts/function_lib
rary.R")
library(Seurat)
library(glue)
library(dplyr)
library(qs)
set.seed(42)

# Read in dataset
path <-
"/home/nic002/nic002_csd922_project/bmt_sc_dataset_processing/DGE/dataset_seu_7_29.3.fina
lized"
seu <- qread(glue("{path}/seu.donor.qs"))
# DGE schema
dataset_var <- "donor"
split_var <- 'lineage_celltypes'
```

```
ident_var <- "patient"
mode_var <- "vs_all"
contrasts <- make_contrasts(seu,
                            ident_var=ident_var,
                            mode=mode_var)
# Wilcox
test.use <- 'wilcox'
schema_name <-
glue("DGE1_{test.use}_Dataset_{dataset_var}_Split_{split_var}_Groupby_{ident_var}_Mode_{m
ode_var}")
out_path <- glue("{path}/{schema_name}/")
dir.create(path = out_path, recursive=T)
run_dge_split_modular(seu=seu,
                      split_vars=split_var,
                      ident_var=ident_var,
                      contrasts=contrasts,
                      out_path=out_path,
                      test.use=test.use)
#### END
```

- **Example script (shell) for running requesting 32 CPUs and 128G RAM**

```
#### File name: run_example.sh
#### START
#!/bin/bash
#SBATCH --mail-user=nic002@ucsd.edu
#SBATCH --job-name="run_example"
#SBATCH --output="run_example.%j.%N.out"
#SBATCH --error="run_example.%j.%N.err"     # Standard error file
#SBATCH --partition=shared
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=32
#SBATCH --mem=128G
#SBATCH --account=csd922
#SBATCH --export=ALL
#SBATCH -t 08:00:00

module reset
module load cpu/0.17.3b
module load gcc/10.2.0
module unload anaconda3/2020.11

# Activate your Conda environment containing an R installation (see IRkernal Conda
Environment section above for environment setup)
module load slurm
source ~/.bashrc
source ~/miniconda3/etc/profile.d/conda.sh && conda activate

conda activate bio_r_py
cmd="Rscript example.R"
```

```
#### END
```

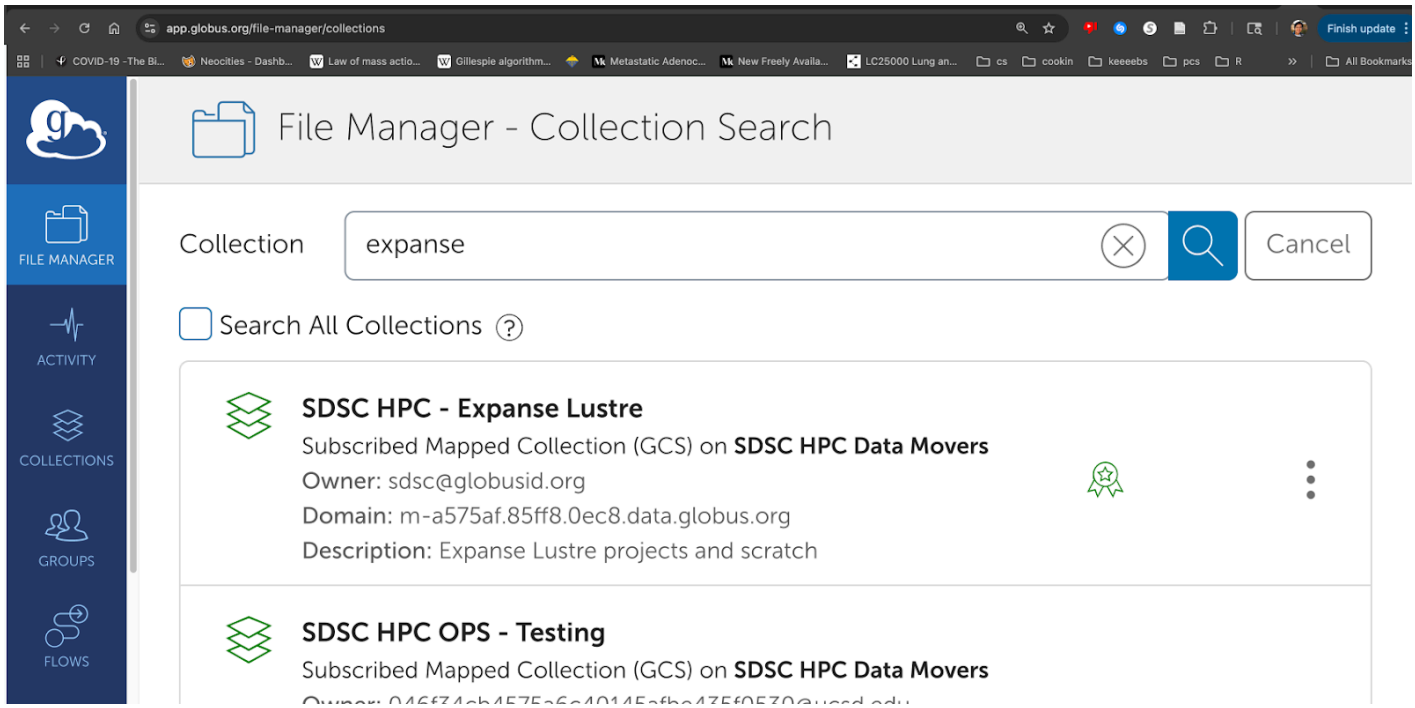
- **Submit script to sbatch to request a job**

```
# Make sure that your .R and .sh files are in known locations
(base) [nic002@exp-9-55 ~]$ ls
> example.R run_example.sh

# Submit job request with sbatch
(base) [nic002@exp-9-55 ~]$ sbatch run_example.sh
```

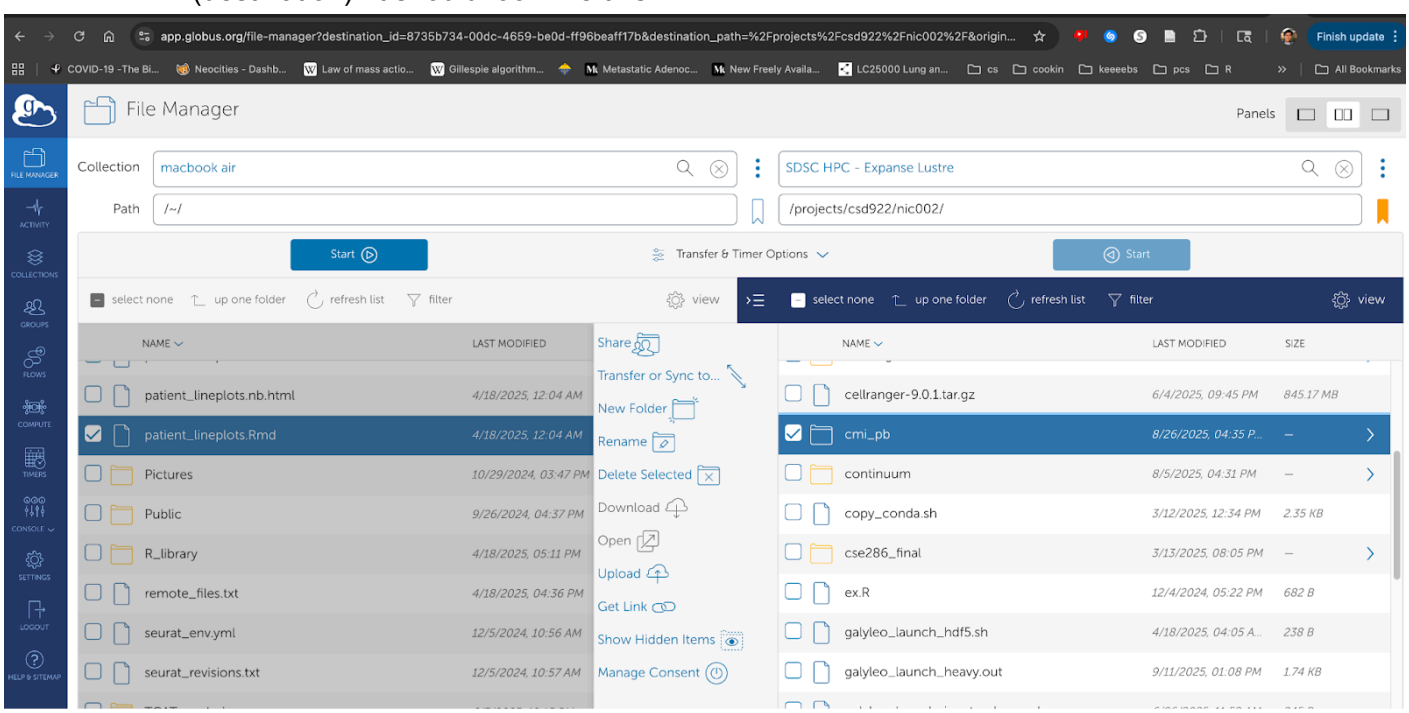
X. File Transfer

- **Globus (GUI)**:
  - Excellent way to schedule/facilitate large data transfers to and from the server
  - Typical globus login page:

  - Once logged in, add Expanse as an "endpoint"
    - End points are addresses registered in Globus that correspond to computer addresses (eg Expanse or even your own laptop)
    - **You can search for the expanse end point by entering it into the "Collection" search bar:**



- **<u>KEY STEP</u>: To add your own computer as an endpoint, download [Globus Connect Personal]**
- To begin file transfer (sending a file from laptop to Expanse) go to the File Manager tab and select your laptop end point on the left (source) and the expanse end point on the right (destination). It should look like this:



- After clicking start, you should get a notification that the transfer request submitted successfully/unsuccessfully

- **SFTP (CLI)**:
  - Secure file transfer protocol (SFTP) plays much like SSH
  - Can be used to move files from one computer to another using the **get** and **put** commands

```
### Using put to send a file to server
# I am currently in my mac and want to transfer a pdf and a folder of files over to the
server
(base) nicholaschan@Nicholass-MacBook-Air-2 ~ % ls
> madagascar2.pdf madagascar_pdfs_dir

# I login to expanse using sftp
(base) nicholaschan@Nicholass-MacBook-Air-2 ~ % sftp nic002@login.expanse.sdsc.edu

# Once logged in, I can go to a desired directory on expanse
[nic002@login02 nic002]$ cd ~/movies

# I can use regular put to move 1 file
[nic002@login02 nic002]$ put madagascar2.pdf

# I can use put -r to move many files
[nic002@login02 nic002]$ put -r madagascar_pdfs_dir
```

```
### Using get to download a file from server
# I am currently in my mac and want to download a file and a folder of files over to the
server
# I login to expanse using sftp
(base) nicholaschan@Nicholass-MacBook-Air-2 ~ % sftp nic002@login.expanse.sdsc.edu

# Once logged in, I can go to a desired directory on expanse
[nic002@login02 nic002]$ cd ~/movies

# I can use regular get to download 1 file
[nic002@login02 nic002]$ get madagascar2.pdf

# I can use put -r to download many files
[nic002@login02 nic002]$ get -r madagascar_pdfs_dir
```

- **Cyberduck (GUI)**:
  - Globus and sftp are better, but this is aight

[ARCHIVED STUFF]

XI. Jupyter Lab Setup (Harder way from Amariuta lab lol, see Interactive Computing for my easier method)

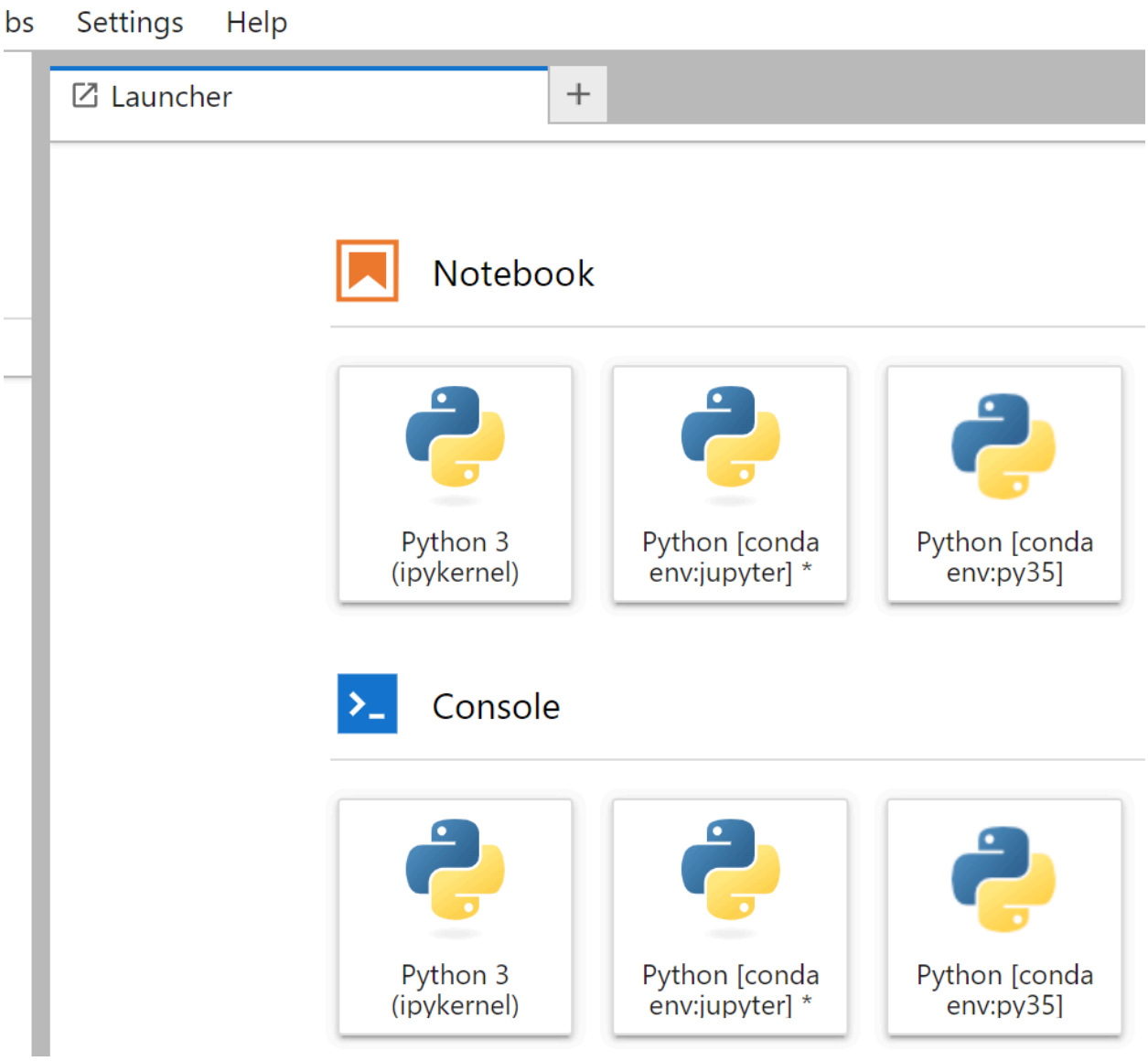
Note: Sections below this were taken from the Amariuta lab

Read carefully and follow the first three session 'Setup Jupyter', 'Setup a conda environment to use with Jupyter', and 'Usage' of this wiki page (under a branch 'hpc-jupyter-setup'). For the following optional part, please do not try to run those codes because they have not been tested by us yet.

Also, if you are having trouble load the localhost:8888, it could result from your own laptop using the port 8888. Please log out the expanse, open terminal and do the following:

```
# kill everything that is using the port 8888
lsof -ti:8888 | xargs kill -9
```

Remember that if you want to have multiple R/Python environments for different work/project which is highly recommended, you need to set up a separate R/Python environment with rkernel/ipykernel installed in each environment. And the nb_conda_kernel will automatically detects all kernel-available R/Python environments and they will be displayed in this page when you open Jupyter Lab.



Default Anaconda Jupyter Notebook (Works pretty well, also easier to setup!)

go to project dir

```
cd /expanse/lustre/projects/ddp412
# and go to your working dir
cd [dir_name]
# load environment module
export PATH="/cm/shared/apps/sdsc/galyleo:${PATH}"
# use galyleo to launch Jupyter
galyleo launch --account ddp412 --partition shared --cpus 1 --memory 2 --time-limit 00:30:00 --env-modules cpu/0.15.4,anaconda3
```

There's more parameters here: https://github.com/mkandes/galyleo

Then open the URL link provided in your browser. The wait time for the Jupyter job to be proxied could be extremely long. For example, I launched a 1GB 15 minute session today and waited for almost 2 hours. But this depends on how busy the server is.

No need to open any node before using galyleo, only in the login node could work.

XII. Commands related to Job, Module, and Packages

A. R packages

Running R on Expanse:
module load cpu/0.15.4
module load gcc/9.2.0
module load r/4.0.2-openblas
Rscript (To check if R is ready)

R packages modules:

module load r-dplyr
module load r-lattice/0.20-38-openblas

Custom R environment

If you ask the help desk to download a specific R package, they will most likely direct you to create a ***custom R environment using anaconda***
Tutorial (to save time, look at the commands I ran below):
https://www.youtube.com/watch?v=UZARwb1LcUA
R packages: https://anaconda.org/r/repo

Commands I ran to install **r environment** and **r package from CRAN**:

```
conda config --add channels bioconda
conda config --add channels conda-forge
conda create -n r_env r-base bioconductor-biocinstaller
conda activate r_env
conda install r-<package name here>
```

Tabix:

```
module load gcc/10.2.0
module load tabix/2013-12-16
```

B. Job Submission

Use 'sbatch ${your_job}.sh' to submit a job.

One example header to put into your job file is:

```
#!/bin/bash
#SBATCH --job-name="cd4 tem meta"
#SBATCH --output="cd4 tem meta"
#SBATCH --partition=shared
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --mem=10G
#SBATCH --account=ddp412
#SBATCH --export=ALL
#SBATCH -t 05:00:00
```

Please always use "shared" node and always set nodes=1

Useful Commands:

- list process running on current node
  ```
  ps aux | grep ${your_user_name}
  ```
- List all running jobs
  ```
  squeue -u ${your_user_name}
  ```
- Cancel a job

```
scancel ${jobid}
```

- Check the service usage situation of the working project

```
expanse-client project csd832
```

- Check your account service usage situation

```
sacct --user=${your_user_name} --start=2023-09-22T09:00:00
```

- Check one job service usage situation

```
sacct --job ${job_id} --format=MaxRSS
```

XIII. Tips and Tricks

ssh key to avoid typing in password every time

Create ssh private and public key pair here:
https://docs.digitalocean.com/products/droplets/how-to/add-ssh-keys/create-with-openssh/

Add your public key to this file in expanse (just copy and paste it):

```
~/.ssh/authorized_keys
```

Alternatively, you can run the following command on your local computer:

```
ssh-copy-id -i /path/to/your/public/key.pub $your_username@login.expanse.sdsc.edu
```

create alias for command you use often

cd ~ (make sure you are in home directory)

vim .bashrc (edit .bashrc file where you will add an alias)

Add an alias to the bottom of this file

alias <alias name>=”<command>”

For example, I created an alias for a debug partition interactive session by adding this line to the file:

alias interactive="srun --partition=debug --pty --account=ddp412 --nodes=1 --ntasks-per-node=1 --mem=2G -t 00:30:00 --wait=0 --export=ALL /bin/bash"

Next time I want to run the command

“srun --partition=debug --pty --account=ddp412 --nodes=1 --ntasks-per-node=1 --mem=2G -t 00:30:00 --wait=0 --export=ALL /bin/bash”

All I have to do is run “interactive” instead

Make sure you logout and log back into expanse before using the alias

Using multiple cores for parallel runs

mpirun -n <number of tasks>

I have not tried this yet, but there are packages that allow you to write for loops so that it executes on multiple cores in parallel

I will update this if I ever try this

Downloading large files to expanse

When you download large files to expanse using wget, it takes FOREVER

Try using aria2 instead! At a high level, it uses multiple channels in parallel to download the contents of a file from the web!

Luckily, it is a module in expanse already!

> module load aria2/1.35.0
> aria2c -s16 -x16 <URL>
^ the above command downloads using 16 channels in parallel

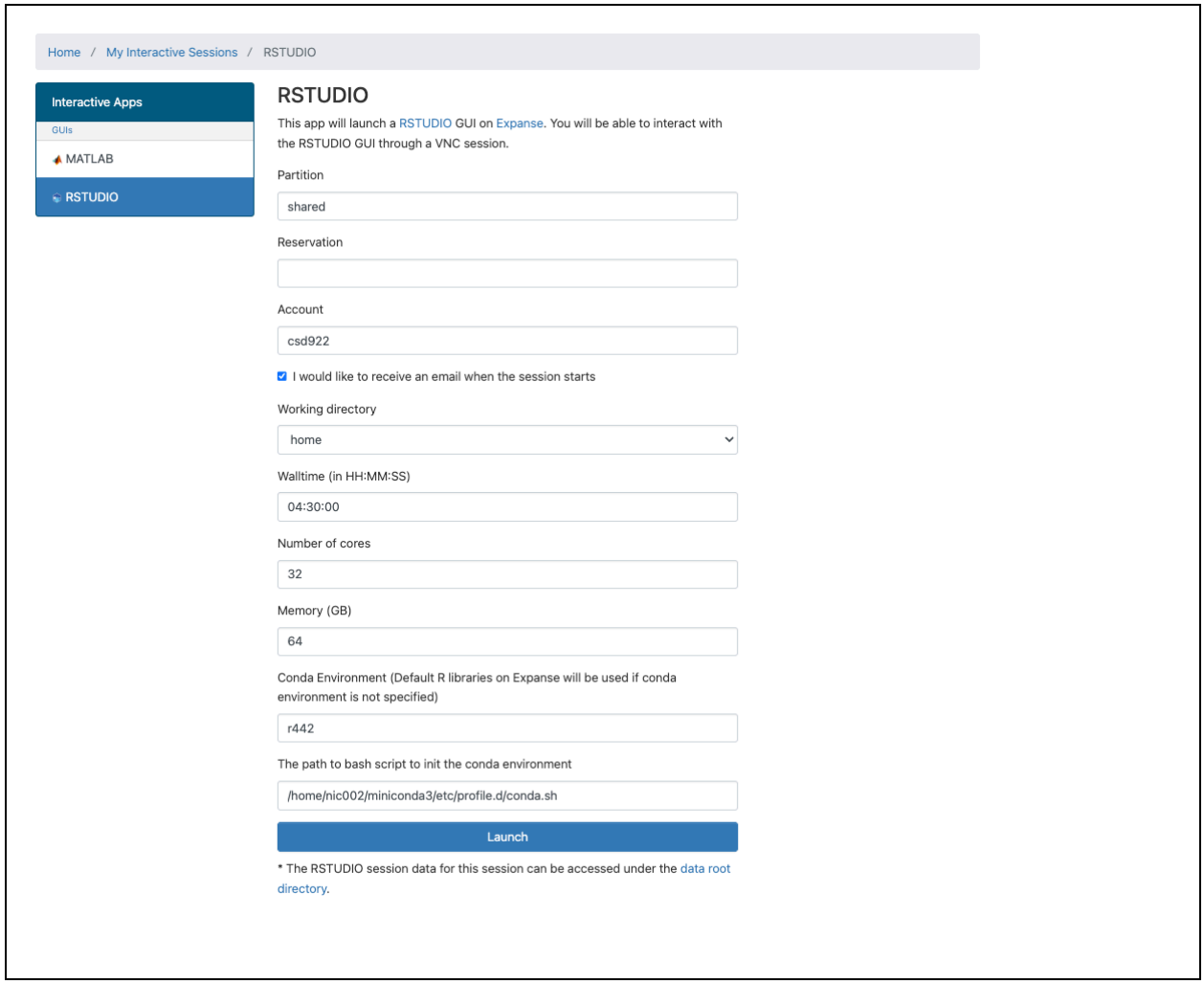
Moving files from your local computer to expanse and vice versa

Do you want to move a large number of files from expanse to your local computer (or vice versa)?
Download https://cyberduck.io/ to directly connect your local file system to expanse

1. Go to https://cyberduck.io/
2. Download for mac or windows, and run the installer
3. Open the cyberduck app
4. Click open connection
5. Change drop down from the top to "SFTP (SSH File Transfer Protocol)"
6. Server: login.expanse.sdsc.edu
7. Username and password just like you are logging into Expanse
8. Use SSH private key if you have it
9. Once you are connected, you can navigate to the expanse/lustre file system and directly download files to your local computer without writing an scp command everytime

RStudio: I usually just start up jobs using the Expanse GUI/online portal - link

- If you are asked for a path to init a conda env, look for something like this: /home/nic002/miniconda3/etc/profile.d/conda.sh
  - Generally try to minimize the amount of things that you download on conda for use with R, instead try to use R's install.packages() command and other options native to R
    - Libraries installed using native R commands like install.packages() likely go to the folder found by running .libPaths()
- Example of what I use to start a session: (zoom in lol)



- **WARNING**: Rstudio runs really really really slow on Expanse, I strongly recommend using R through jupyter lab if you desire a notebook like environment for running code